ZuORDNER – Technische Dokumentation & Code-Handbuch

Version 2.0 – Systemarchitektur, Algorithmen, Dynamic Rule Engine & Frontend/Backend Entwicklerhandbuch

1. Einleitung & Zielsetzung

Die Anwendung **ZuORDNER** löst das komplexe Verteilungsproblem von Teilnehmern auf Gebäude, Räume und Betten in Jugendherbergen und Ferienunterkünften. Bisherige manuelle Planung erfordert oft Stunden und birgt das Risiko von Fehlern (z.B. Senioren auf oberen Hochbetten oder Missachtung von Geschlechtertrennung).

Mit der neuen Architektur wird der gesamte Prozess vollständig automatisiert:

- **Flexible Datenverwaltung:** Beliebige Gebäude, Etagen, Räume und Einzelbetten.
- **Dynamisches Regelsystem:** Anpassung von Harte Verbote und Weiche Präferenzen ohne Code-Änderung.
- **6-Schritt Excel/CSV Import:** Geführter Import mit manuellem Spaltenmapping.
- **Mathematische Optimierung:** Ungarischer Algorithmus (Kuhn-Munkres) als Startlösung, danach lokale Suche, die auch Paare, Gruppen und Geschlechtertrennung berücksichtigt.

> **Neu in Version 2.0:**
>
> - Die Standard-Regeln (Geschlechtertrennung, Paare, Gruppen, Senioren) haben einen festen Schlüssel (`standardKey`) und lassen sich **unabhängig voneinander** schalten. Vorher wurden sie über Wörter im Namen gesucht – der Paar-Schalter hat dadurch die Geschlechter-Regel umgeschaltet.
> - Neue Regeln bekommen vom Server eine eindeutige ID (vorher bekamen alle die leere ID und ließen sich danach nicht mehr umschalten).
> - Regeln für Zimmernachbarn (Paare, Gruppen, Geschlechtertrennung) beeinflussen jetzt wirklich die Verteilung (vorher wurden sie erst nachträglich ausgewertet).
> - Personen ohne Bett werden mit Grund angezeigt; alte Projektdateien werden beim Start automatisch repariert (mit Sicherheitskopie).

2. Systemarchitektur & Data Flow

Das System ist nach dem modernen **Decoupled Client-Server Prinzip** aufgebaut:

> **React 19 / TypeScript Web Frontend** ⟵ (REST / JSON) ⟶ **Spring Boot 3 Java Backend**

Schicht	Technologie	Aufgabe
Frontend UI	React, TS, Vite, TailwindCSS	Geführte Benutzeroberfläche, Regel-Editor, Import-Wizard, Belegungsplan.
REST API Layer	Spring Boot Web Controllers	Bereitstellung von JSON-Endpunkten für CRUD-Operationen & Berechnung.
Rule Engine	Java Condition Engine (läuft ab Java 17)	Bewertung von Personen, Räumen & Betten gegen dynamische WENN-Bedingungen.
Optimization Core	Hungarian Solver + LocalSearchOptimizer	Startlösung per Kuhn-Munkres, danach Verbesserung mit Simulated Annealing und Hill Climbing (inkl. Zimmernachbar-Regeln).
Import Engine	Apache POI, OpenCSV, TableMappingService	Einlesen von Excel/CSV Dateien & Aufbereitung im Mapping-Wizard.
Persistence	Jackson JSON Repository	Dateibasierte Speicherung der Projekte unter `data/projects/`.

3. Die Algorithmen im Detail

A. Dynamic Rule Engine (Bewertungslogik)

Die `RuleEngine` berechnet für jedes Person-Bett-Paar einen numerischen Eignungswert (Score).

> **Score-Formel:**
> `Score(Person i, Bett j) = ∑ (Weiche Punkte) - (Harte Regelverletzung ? 999999 : 0)`

Es wird zwischen zwei Regeltypen unterschieden:

- **Harte Regel (HARD / FORBID)**: Bei Verletzung wird der Score auf `-999999` gesetzt. Eine solche Zuordnung wird nie gewählt – lieber bleibt eine Person ohne Bett (mit Begründung im Dashboard).
- **Weiche Regel (SOFT / PREFERENCE)**: Verleiht Punkteboni (z.B. `+50`) oder Punktabzüge (`-20`).

Zusätzlich unterscheidet der Regel-Bezug (`TargetScope`):

- `BED_PERSON` / `ROOM_PERSON`: eine Person und ihr Bett bzw. Raum (z.B. Senioren nicht ins obere Hochbett).
- `GENDER_SEPARATION`: Männer und Frauen nicht im selben Zimmer; markierte Mädchen-/Jungenzimmer nur für das jeweilige Geschlecht. Ausnahme: Paare, wenn die Paar-Regel aktiv ist. Divers/unbekannt wird nicht eingeschränkt.
- `PAIR_CO_LOCATION`: Paare (gleiche Paar-ID) zusammen, +50 Punkte pro Person, +10 im Doppelbett.
- `GROUP_CO_LOCATION`: Gruppen zusammen, +20 Punkte pro Gruppenmitglied im selben Zimmer.
- `CONDITION_CO_LOCATION`: alle Personen, auf die die WENN-Bedingungen zutreffen (z.B. Alter > 65), zusammen.

B. Der Ungarische Algorithmus (Hungarian Solver)

Um die global beste Verteilung aller Teilnehmer zu finden, nutzt das System den **Kuhn-Munkres Algorithmus** mit einer Laufzeit von $O(N^3)$.

1. **Standardisierung:** Wenn Personen-Anzahl N ≠ Betten-Anzahl M, wird die Matrix durch Dummy-Elemente auf K x K aufgestockt (K = max(N, M)).
2. **Score-Invertierung:** Da der Standard-Hungarian ein Minimierungsproblem löst, werden die Scores umgewandelt:

`Cost[i][j] = MaxScore - Score[i][j]` (Verbotene Felder erhalten `Cost = 1.000.000`).

3. **Augmentierende Pfade:** Die ungarische Reduktion findet in Millisekunden die Zuordnung, die die Gesamtsumme der Punkte aller Personen maximiert.

Grenze: Der Ungarische Algorithmus bewertet jede Person nur zusammen mit IHREM Bett. Wer im selben Zimmer landet, kann er nicht berücksichtigen. Seit Version 2.0 ist sein Ergebnis deshalb nur der Startpunkt.

C. Lokale Suche (LocalSearchOptimizer, neu in 2.0)

Ausgehend von der Startlösung werden sehr viele kleine Änderungen ausprobiert (Person in freies Bett, zwei Personen tauschen, Person vorerst ohne Bett):

> **Gesamtpunktzahl:**
> `Σ Punkte(Person, Bett) + Σ Punkte(Zimmernachbarn) − 100.000 × (Personen ohne Bett)`
> Harte Verstöße zählen −1.000.000. Rangfolge: keine Verstöße > möglichst alle unterbringen > möglichst viele Wunsch-Punkte.

1. **Simulated Annealing:** Verbesserungen werden immer angenommen, kleine Verschlechterungen anfangs mit einer Wahrscheinlichkeit, die mit der "Temperatur" sinkt. So bleibt die Suche nicht in Sackgassen stecken. 20.000 Versuche pro Person (höchstens 10 Millionen).
2. **Hill Climbing:** Am Ende werden systematisch alle Umzüge/Tausche geprüft, bis keine Verbesserung mehr möglich ist.
3. **Deterministisch:** Fester Zufalls-Startwert – gleiche Eingabe ergibt gleiches Ergebnis.

Gemessen: ca. 0,1 s für 30 Personen, 0,4 s für 100 Personen, 0,8 s für 200 Personen.

4. Entwickler-Handbuch: Wichtigste Klassen im Backend

1. `RuleEngine.java`

Verantwortlich für das Einlesen und Auswerten aller WENN-Bedingungen. Extrahiert dynamisch Attribute wie `person.age`, `person.gender`, `room.girlsRoom`, `bed.isTopBunk`.

2. `HungarianSolver.java`

Rein mathematische Solver-Klasse. Wandelt die Maximierungsmatrix in ein Min-Cost Problem um und führt den Kuhn-Munkres Algorithmus aus.

3. `AssignmentService.java`

Orchestriert den Ablauf:

```
1. Alle Betten und Räume einsammeln
2. Tabelle Person x Bett aus der RuleEngine befüllen
3. Tabelle Person x Person (Zimmernachbarn: Paare, Gruppen, Geschlechtertrennung)
4. HungarianSolver.solveMaxWeightMatching() als Startlösung
5. LocalSearchOptimizer.optimize() verbessert die Verteilung
6. ScoreExplanation-Liste pro Person + Grund für jede Person ohne Bett
```

4. `LocalSearchOptimizer.java` (neu in 2.0)

Verbessert die Startlösung (siehe Abschnitt 3C). Berechnet Punkte-Änderungen nur für die betroffenen Räume (`delta()`) und ist dadurch sehr schnell.

5. `StandardRule.java` & `ProjectMigration.java` (neu in 2.0)

`StandardRule` definiert die 4 Standard-Regeln mit festem Schlüssel. Der Endpunkt `PUT /api/projects/{id}/rules/standard/{key}` schaltet genau eine davon an oder aus. `ProjectMigration` repariert beim Start alte Projektdateien (leere/doppelte Regel-IDs, alte Geschlechter-Regel) und legt vorher eine Sicherheitskopie `*.json.v1-backup` an.

6. `TableMappingService.java`

Entkoppeltes Interface zur Transformation von Rohdaten-Tabellen in strukturierte `Person` -Objekte. Ermöglicht den schnellen Austausch gegen eine KI-gestützte Spaltenerkennung in der Zukunft.

5. Frontend-Komponenten im Überblick

Komponente	Datei	Funktion & Besonderheiten
BuildingManager	`BuildingManager.tsx`	Gebäude anlegen/löschen, Räume konfigurieren, einzelne Betten beliebiger Bettart hinzufügen und entfernen.
ParticipantManager	`ParticipantManager.tsx`	Teilnehmerliste durchsuchen, filtern, manuell anlegen, bearbeiten oder löschen.
ImportWizard	`ImportWizard.tsx`	6-stufiger Assistent für Excel/CSV Dateien mit Spaltenzuordnungs-GUI und Validierungsanzeige.
RuleEditor	`RuleEditor.tsx`	4 unabhängige Standard-Schalter (über `standardKey`) mit Hinweis zum Zusammenspiel, plus Baukasten für eigene WENN-DANN Regeln mit Details-Ansicht.
AssignmentDashboard	`AssignmentDashboard.tsx`	Belegungsplan-Visualisierung mit Punkten, Erklärungs-Fenster für jede Person und Liste der Personen ohne Bett (mit Grund).
ProjectSelectorModal	`ProjectSelectorModal.tsx`	Projektwechsel, Erstellung neuer Projekte und dauerhaftes Löschen von Projekten.

6. Zusammenfassung & Ausblick

Die Anwendung ist voll einsatzbereit, schnell (unter einer Sekunde für bis zu 200 Personen) und extrem flexibel. Dank der sauberen Trennung von Frontend, REST API, Rule Engine und Import Pipeline kann das System in Zukunft problemlos erweitert werden (z.B. Anbindung an Google Sheets API oder ein KI-Modell zur automatischen Spaltenerkennung).